STQA INTERNSHIP PROJECT REPORT

Cohort: Real-Time Social Communication Platform with AI Assistance

Structured as per the STQA flow shared by the student. Replace the bracketed fields with final official details where required.

STUDENT NAME **[Student Name]**	ENROLLMENT NUMBER **[Enrollment Number]**
GUIDE INFO **[Guide Name]**	SUBMITTED TO **[HOD / Department / College Name]**
COMPANY / ORGANIZATION **[Company Name]**	ACADEMIC YEAR **2025-2026**

This report is based on the actual Cohort codebase: React + Vite frontend, Express + Socket.IO backend, MongoDB database, JWT authentication, and Gemini-powered AI assistance.

College Certificate

Reserved for Official College Certificate

Attach the signed college certificate on this page.

Log Sheet

Reserved for Log Sheet

Insert the approved internship log sheet on this page.

Company Certificate

Reserved for Company Certificate

Attach the company internship / completion certificate on this page.

Acknowledgement

I express my sincere gratitude to **[College Name]**, my guide **[Guide Name]**, and the Department of **[Department Name]** for their constant support and guidance during the internship and project work.

I am also thankful to **[Company Name]** for providing the opportunity to work on a practical full-stack software project. The experience helped me understand frontend-backend integration, realtime communication, database modeling, and software documentation in a professional context.

Finally, I thank my peers and family members for their encouragement and support during the successful completion of this project and report.

Index

SR. NO.	CONTENTS	PAGE NO.
1	Project Title, Guide Info, Student Info, Submitted To	1
2	College Certificate	2
3	Log Sheet	3
4	Company Certificate	4
5	Acknowledgement	5
6	Index	6
7	Company Profile	7
8	Project Profile	8
9	Existing System	9
10	Proposed System	10
11	Tools and Technology	11
12	System Flow Diagram	12
13	Use Case, Class, Activity, Sequence / DFD	13-16
14	Class / ER Diagram	17
15	Data Dictionary	18-19
16	Input Screen Designs	20
17	Conclusion	21
18	Bibliography	22

Company Profile

COMPANY NAME

[Company Name]

WORK DOMAIN

Software development, web application engineering, and product-oriented implementation.

INTERNSHIP EXPOSURE

Frontend UI work, backend route development, realtime logic, database modeling, and debugging.

PROFESSIONAL RELEVANCE

Hands-on exposure to current full-stack development tools and real project workflows.

If the college requires a detailed official company profile, this page can be updated with the exact organization history, services, and contact details provided by the company.

Project Profile

PROJECT TITLE

Cohort - Real-Time Social Communication Platform

PROJECT TYPE

Full-stack web application

OBJECTIVE

To create a unified platform for messaging, communities, events, updates, and AI interaction.

TARGET USERS

Students and general users who need social communication and collaboration features in one app.

Main Modules

- Authentication and onboarding
- Direct and group chat
- Communities and groups
- Events and expiring updates
- AI assistant and translation

Existing System

Observed Limitations

- Users often depend on different apps for chat, updates, and event coordination.
- Traditional academic messaging projects lack advanced realtime behavior.
- AI support and translation are usually absent.

Need for Improvement

- Single platform for communication and coordination.
- Better social discovery and group management.
- Modern user experience with live updates and richer interaction.

Proposed System

Cohort is proposed as an integrated communication platform where the same user can manage contacts, chat in realtime, participate in communities, create events, post updates, and use AI assistance without leaving the application ecosystem.

Realtime

Typing indicators, read receipts, online status, and socket-based updates.

Social

Friends, communities, groups, event participation, and personal updates.

Intelligent

AI chat and multilingual translation for better accessibility.

The proposed system improves both usability and technical structure by separating UI, APIs, realtime events, and data storage.

Tools and Technology

LAYER	TECHNOLOGY	PURPOSE
Frontend	React 19, Vite, React Router DOM	Responsive SPA interface and route management
Client Utilities	Axios, Socket.IO Client, Lucide React, Emoji Picker, Three.js	API calls, realtime events, icons, emoji support, and UI visuals
Backend	Node.js, Express 5	REST APIs, middleware handling, and server logic
Realtime	Socket.IO	Live message delivery, typing, online status, read receipts, call signaling
Database	MongoDB Atlas, Mongoose	Schema-based document persistence
Security	JWT, bcryptjs	Authentication and password hashing
Media	Multer	Upload handling for profile and media assets
AI	Gemini API	AI chat and translation support

System Flow Diagram

```
User
  |
  v
React Frontend
  |---- REST APIs ----> Express Backend ----> MongoDB Atlas
  |
  |---- Socket.IO ----> Realtime Server
  |
  |---- AI Request ---> Gemini API

Flow Summary:
1. User performs action in frontend.
2. Frontend sends API requests to backend.
3. Backend stores / fetches data from MongoDB.
4. Realtime events are delivered through Socket.IO rooms.
5. AI requests are forwarded to Gemini and returned to UI.
```

UML Diagram - Use Case Diagram

```
Actors:
- User
- Admin

Primary Use Cases:
- Register / Login
- Manage Profile
- Search Users / Add Contacts
- Start Direct Chat / Group Chat
- Send Message / React / Reply / Read
- Join Community / Manage Group
- Create Event / RSVP Event
- Post Update
- Use AI Assistant / Translate Messages

Admin-Specific:
- Approve join requests
- Moderate community / group settings
- Manage announcement or admin-only spaces
```

UML Diagram - Class Diagram

```
User
- name, username, email, password
- profilePic, bio, interests
- friends[], incomingFriendRequests[], outgoingFriendRequests[]

Chat
- isGroup, kind, participants[], admin, lastMessage

Message
- sender, chat, content, type, attachments[], readBy[], reactions[]

Community
- name, description, category, creator, admins[], members[], announcementChat

Group
- name, description, creator, admins[], members[], chat, community, settings

Event / Update
- creator or author, date or expiresAt, attendees or visibility

Relations:
User -> Chat -> Message
User -> Community -> Group
User -> Event
User -> Update
```

UML Diagram - Activity Diagram

```
Start
  ->
Login / Authenticate
  ->
Open Dashboard
  ->
Select Module
  -> Chat -> Send message / read / react
  -> Community / Group -> join / manage / approve
  -> Event -> create / RSVP
  -> Update -> create / view
  -> AI -> ask / translate
  ->
Store / Broadcast / Display result
  ->
End
```

UML Diagram - Sequence Diagram / DFD

```
Sequence Example: Message Sending
User -> Frontend: enter content
Frontend -> Backend: POST /api/message
Backend -> MongoDB: save message
Backend -> MongoDB: update chat.lastMessage
Backend -> Socket.IO room: emit receive-message
Backend -> Frontend: return created message

DFD Level-0
User Inputs -> Cohort System -> Database
User Inputs -> Cohort System -> AI Service
User Inputs -> Cohort System -> Realtime Notifications
```

Class / ER Diagram

```
Entities:
User, Chat, Message, Community, Group, Event, Update

Core Relationships:
User 1..* -> Chat
Chat 1..* -> Message
User 1..* -> Message
Community 1..* -> Group
Community 1..* -> Member(User)
Group 1..1 -> Chat
User 1..* -> Event
User 1..* -> Update

This ER structure supports social relationships, realtime communication,
community organization, and short-lived content sharing.
```

Data Dictionary - Part I

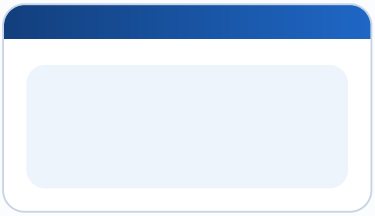
ENTITY	FIELD	TYPE	DESCRIPTION
User	name	String	Display name of the user
User	username	String	Unique identity used for login and mentions
User	email	String	Unique email for authentication
User	friends[]	ObjectId[]	Connected contact references
Chat	isGroup	Boolean	Direct or group chat marker
Chat	kind	Enum	direct, group, or community-announcement
Chat	participants[]	ObjectId[]	Users in the chat
Message	sender	ObjectId	User who sent the message
Message	chat	ObjectId	Related chat room
Message	content	String	Message body

Data Dictionary - Part II

ENTITY	FIELD	TYPE	DESCRIPTION
Message	attachments[]	Object[]	Metadata for uploaded files
Message	reactions[]	Object[]	Emoji reactions with user references
Community	announcementChat	ObjectId	Official community announcement channel
Group	settings	Object	Admin-only, invite, and approval controls
Event	date / endDate	Date	Event schedule
Event	attendees[]	ObjectId[]	Users who RSVP
Update	visibility	Enum	public, contacts, community, or group
Update	expiresAt	Date	24-hour auto-expiry timestamp

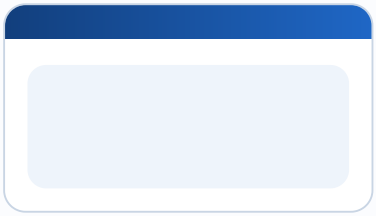
Input Screen Designs

Login Screen



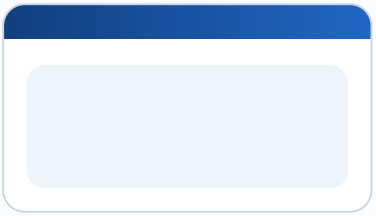
Email/password based secure sign in.

Signup Screen



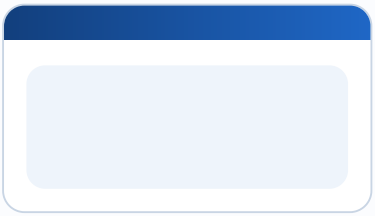
Registration and onboarding information capture.

Home Dashboard



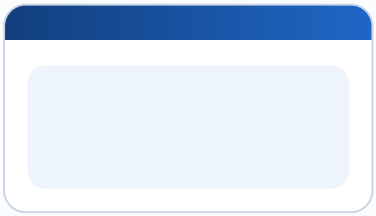
Central workspace for chats, AI, communities, groups, updates, and events.

Chat Window



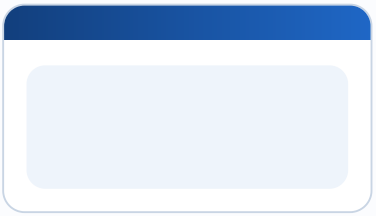
Message list, search, scheduling, reactions, and realtime conversation area.

Community / Group View



Membership, moderation, invite, and announcement controls.

Settings / Profile



Personal information, avatar, education, interests, and preferences.

Replace these conceptual panels with actual screenshots if your college prefers implementation screenshots.

Conclusion

The Cohort project demonstrates a complete full-stack communication platform that combines realtime messaging, social organization, events, updates, and AI assistance into a single system.

From a learning perspective, the project provided practical exposure to React-based UI development, Node.js / Express API design, Socket.IO realtime workflows, MongoDB schema design, authentication, and documentation.

The architecture is modular enough for future improvements such as automated tests, deployment hardening, media enhancements, and stronger production call support.

Bibliography

1. React Documentation - https://react.dev
2. Vite Documentation - https://vite.dev
3. Express.js Documentation - https://expressjs.com
4. Socket.IO Documentation - https://socket.io/docs
5. MongoDB Documentation - https://www.mongodb.com/docs
6. Mongoose Documentation - https://mongoosejs.com/docs
7. Axios Documentation - https://axios-http.com
8. JWT Reference - https://jwt.io
9. Gemini API reference used for AI integration
10. Cohort project source files and project documentation from the codebase